SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	A. Mohith krishna	**Reg. No.:**	192425199
Section / Batch:	2024	**Date:**	15-08-26

Code of Conduct

I A.Mohith Krishna(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Mohith

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights. The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**
- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Task 1 – N-gram Based Text Prediction

code:

```
from collections import Counter
text = """
artificial intelligence is changing the world
artificial intelligence is useful
artificial intelligence is growing
machine learning is useful
machine learning can solve problems
machine learning can predict results
"""
words = text.lower().split()
unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
trigram = Counter(zip(words, words[1:], words[2:]))
sentence = input("Enter incomplete sentence: ").lower().split()
n = int(input("Enter N (1, 2 or 3): "))
predictions = []
for word in unigram:
    if n == 1:
        probability = unigram[word] / len(words)
    elif n == 2:
        count = bigram[(sentence[-1], word)]
        probability = count / unigram[sentence[-1]] \
            if unigram[sentence[-1]] else 0
    else:
        count = trigram[(sentence[-2], sentence[-1], word)]
        probability = count / bigram[
            (sentence[-2], sentence[-1])
        ] if bigram[
            (sentence[-2], sentence[-1])
        ] else 0
    if probability > 0:
        predictions.append((word, probability))
predictions.sort(key=lambda x: x[1], reverse=True)
print("\nTop 5 Predictions:")
for word, probability in predictions[:5]:
    print(word, "=", round(probability, 3))
```

Sample Output:

```
Enter incomplete sentence: artificial intelligence is
Enter N (1, 2 or 3): 3

Top 5 Predictions:
changing = 0.333
useful = 0.333
growing = 0.333
```

Task 2 – Simple Backoff and Interpolation

Code:

```
from collections import Counter
text = """
machine learning can solve problems
machine learning can predict results
machine learning is useful
artificial intelligence can solve problems
artificial intelligence can improve systems
"""
words = text.lower().split()
unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
trigram = Counter(zip(words, words[1:], words[2:]))
sentence = input("Enter sentence: ").lower().split()
def unigram_prob(word):
    return unigram[word] / len(words)
def bigram_prob(w1, w2):
    if unigram[w1] == 0:
        return 0
    return bigram[(w1, w2)] / unigram[w1]
def trigram_prob(w1, w2, w3):
    if bigram[(w1, w2)] == 0:
        return 0
    return trigram[(w1, w2, w3)] / bigram[(w1, w2)]
print("\nBACKOFF PREDICTIONS")
result = []
for word in unigram:
    # First try trigram
    p = trigram_prob(
        sentence[-2],
        sentence[-1],
        word
    )
    # Then bigram
    if p == 0:
        p = bigram_prob(sentence[-1], word)
    # Finally unigram
    if p == 0:
        p = unigram_prob(word)
    result.append((word, p))
result.sort(key=lambda x: x[1], reverse=True)
for word, p in result[:5]:
    print(word, "=", round(p, 3))
print("\nINTERPOLATION PREDICTIONS")
result = []
for word in unigram:
    p1 = unigram_prob(word)
    p2 = bigram_prob(sentence[-1], word)
    p3 = trigram_prob(
```

```
        sentence[-2],
        sentence[-1],
        word
    )
    # weights
    p = 0.2 * p1 + 0.3 * p2 + 0.5 * p3
    result.append((word, p))
result.sort(key=lambda x: x[1], reverse=True)
for word, p in result[:5]:
    print(word, "=", round(p, 3))
```

Sample Output:

```
Enter sentence: machine learning can

BACKOFF PREDICTIONS
solve = 0.667
predict = 0.333
is = 0.143
useful = 0.143
problems = 0.071

INTERPOLATION PREDICTIONS
solve = 0.402
predict = 0.235
is = 0.086
useful = 0.071
problems = 0.036
```

Task 3 – Simple Entropy Evaluation

Code:

```
from collections import Counter
import math
train = """
the cat sits on the mat
the cat eats food
the dog sits on the floor
the dog eats food
the student reads a book
"""
test = [
    "the cat sits on the mat",
    "the quantum processor redesigned the system"
]
words = train.split()
unigram = Counter(words)
bigram = Counter(zip(words, words[1:]))
total = len(words)
```

```
def entropy(sentence):
    test_words = sentence.split()
    value = 0
    count = 0
    for i in range(len(test_words)):
        if i == 0:
            probability = unigram[test_words[i]] / total
        else:
            previous = test_words[i - 1]
            if bigram[(previous, test_words[i])] > 0:
                probability = (
                    bigram[(previous, test_words[i])]
                    / unigram[previous]
                )
            else:
                probability = 0
        if probability > 0:
            value += -math.log2(probability)
            count += 1
    if count == 0:
        return float("inf")
    return value / count
print("ENTROPY EVALUATION")
for sentence in test:
    h = entropy(sentence)
    print("\nSentence:", sentence)
    if h == float("inf"):
        print("Entropy: Infinity")
        print("High Predictive Uncertainty")
    else:
        print("Entropy:", round(h, 3))
        if h > 3:
            print("High Predictive Uncertainty")
        else:
            print("Low Predictive Uncertainty")
```

Sample Output:

ENTROPY EVALUATION

Sentence: the cat sits on the mat
Entropy: 1.584
Low Predictive Uncertainty

Sentence: the quantum processor redesigned the system
Entropy: 3.821
High Predictive Uncertainty

Task 4 – Simple Comparative POS Tagging

Code:

```
# Simple Rule-Based POS Tagger
def rule_based(sentence):
    words = sentence.lower().split()
    result = []
    nouns = ["book", "ticket", "cat", "dog"]
    verbs = ["read", "run", "eat", "book"]
    pronouns = ["i", "you", "he", "she", "we", "they"]
    determiners = ["a", "an", "the"]
    for word in words:
        word = word.strip(".,!?")
        if word in pronouns:
            tag = "PRP"
        elif word in determiners:
            tag = "DT"
        elif word in verbs:
            tag = "VB"
        elif word in nouns:
            tag = "NN"
        elif word.endswith("ing"):
            tag = "VBG"
        elif word.endswith("ly"):
            tag = "RB"
        else:
            tag = "NN"
        result.append(word + "/" + tag)
    # Context rule
    for i in range(len(result) - 1):
        if result[i] == "i/PRP":
            word = result[i + 1].split("/")[0]
            if word == "book":
                result[i + 1] = "book/VB"
    return result
# Simple Stochastic Tagger
dictionary = {
    "i": "PRP",
    "book": "NN",
    "read": "VB",
    "a": "DT",
    "the": "DT",
    "ticket": "NN",
    "cat": "NN"
}
def stochastic(sentence):
    words = sentence.lower().split()
    result = []
    for word in words:
        word = word.strip(".,!?")
        if word in dictionary:
            tag = dictionary[word]
```

```
        else:
            tag = "NN"
        # Context
        if word == "book" and len(result) > 0:
            if result[-1].endswith("/PRP"):
                tag = "VB"
        result.append(word + "/" + tag)
    return result
# Transformation-Based Tagger
def transformation_based(sentence):
    result = rule_based(sentence)
    for i in range(len(result) - 1):
        if result[i] == "i/PRP":
            word = result[i + 1].split("/")[0]
            if word == "book":
                result[i + 1] = "book/VB"
    return result
# Test
sentences = [
    "I book a ticket",
    "I read a book"
]
for sentence in sentences:
    print("\nSentence:", sentence)
    print("Rule-Based:")
    print(" ".join(rule_based(sentence)))
    print("Stochastic:")
    print(" ".join(stochastic(sentence)))
    print("Transformation-Based:")
    print(" ".join(transformation_based(sentence)))
```

Sample Output:

```
Sentence: I book a ticket
Rule-Based:
i/PRP book/VB a/DT ticket/NN
Stochastic:
i/PRP book/VB a/DT ticket/NN
Transformation-Based:
i/PRP book/VB a/DT ticket/NN
Sentence: I read a book
Rule-Based:
i/PRP read/VB a/DT book/NN
Stochastic:
i/PRP read/VB a/DT book/NN
Transformation-Based:
i/PRP read/VB a/DT book/NN
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	**25**	**23**	**15**	**12**	**7**		**82**
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____